

버튼·LCD 통합 실습

VS Code 사전 시뮬레이션 → 오픈소스 CLI → 보드 실험

같은 RTL에 10개 회로를 넣고 버튼으로 선택한다. LCD에 모드 번호와 회로 이름을 표시한다.

작성일 2026. 09. 10. · 이해리

1. 먼저 2,560개 입력과 버튼·LCD 동작을 검사한다.
2. 합성·배치배선·비트스트림을 생성한다.
3. 보드에서 모드를 순환하고 사진·영상으로 설명한다.

통합 실습 목차

1부 · VS Code 사전 시뮬레이션
clone·새 창·workspace·실행·파일형

2부 · 통합 회로 이해
10개 모드·버튼·LCD·보드 연결

3부 · 구현과 결과
CLI 설치·실행·검증 범위
보드·사진·영상·실험 후 레포트

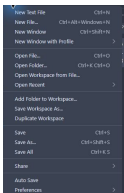
전체 목차 PDF

템플릿과 새 창

Git·Python 3·VS Code와 Icarus Verilog·Yosys·openXC7을 준비한다.

```
git clone https://github.com/Glaysia/fpga-lab-template.git
```

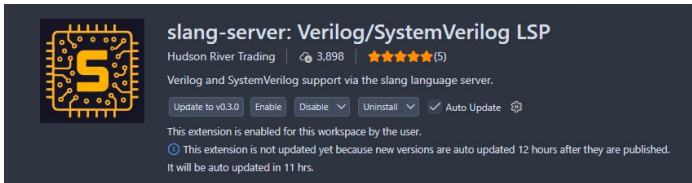
VS Code: File → New Window → Open Workspace from File...



opensource_cli/integrated/opensource_cli_integrated.code-workspace

확장과 Explorer

Extensions에서 slang-server와 VaporView를 Install 또는 Enable (Workspace).

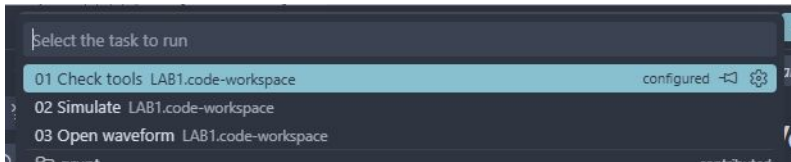


Explorer에서 COMMON → rtl → lab1_integrated.v를 두 번 누른다.

COMMON → tb → tb_lab1_integrated.sv, constraints → lab1_integrated.xdc도 연다.

PROJECT에는 project.json·workspace·결과 폴더가 있다. File → Save All.

세 작업을 순서대로 실행



1. Terminal → Run Task... → 01 Check tools.
 2. 02 Simulate → 종료까지 기다린다.
 3. PROJECT/build/vscode/simulation.log를 연다.
 4. LAB1_PASS lab1_integrated cases=2560을 확인한다.
 5. 03 Open waveform으로 wave.vcd를 연다.
- CLI workspace는 Icarus Verilog를 선택한다.

통합 테스트의 검사 범위

10개 모드 × DIP 256개 = 2,560개 출력 비교.

reset → MODE 01, 짧은 바운스 무시, 길게 눌러도 한 번만 전환, 10→01 순환을 검사한다.

LCD 초기 명령과 외부 버스에서 완성된 두 줄의 번호·이름을 비교한다.

TB는 10ns 클록, 디바운스 4클록, 전원 대기 5클록으로 시간을 줄인다.
실제 합성은 1kHz 보드 설정이다.

XSim 검사 종료: 72430ns. 실제 보드의 전압·LCD 연결은 별도로 확인한다.

VaporView에서 통합 파형 읽기

1. wave.vcd가 텍스트면 탭 우클릭 → Reopen Editor With... → VaporView.
2. Netlist에서 tb_lab1_integrated를 펼친다.
3. clk, rst, mode_button, sw, led, seg_data를 추가한다.
4. dut 안의 mode와 lcd_e, lcd_rs, lcd_rw, lcd_data를 추가한다.
5. Zoom to Fit 뒤 모드가 바뀌는 구간을 확대한다.
6. 버튼 입력부터 모드 변경까지의 지연과 LCD 한 화면 갱신을 구분한다.

실험 전 레포트에 넣을 것

1. 모드별 예상 입력·출력과 비트 순서를 표로 작성한다.
2. 10개 회로·버튼·LCD 파일의 역할을 설명한다.
3. PASS 로그, VCD, 모드 전환 및 LCD 파형 캡처를 연결한다.
4. TB에서 줄인 시간과 실제 1kHz 설정을 구분한다.
5. 실험에서 확인할 버튼·LCD·DIP·LED 장면을 계획한다.

레포트 양식과 예시

통합 구조와 입력 배치

lab1_integrated → 10개 조합회로 + button_onepulse + lcd_modes.

clk=B6(1kHz), KEY1=reset, KEY2=mode_button.

DIP1-8은 sw[7:0], LED1-8은 led[7:0]에 대응한다.

개별 실습의 KEY 입력은 통합본에서 DIP로 옮겨 모드 버튼과 충돌하지 않게 한다.

단일 7세그먼트는 모드 10에서만 동작한다.

통합 동작 명세와 핀표

모드별 조작 1-5

모드	입력	출력
01	a=DIP1, b=DIP2	LED1=AND, 2=OR, 3=XOR
02	a,b,cin=DIP1,2,3	LED1=carry, 2=sum
03	a=DIP1-4, b=DIP5-8	LED1=carry, 2-5=sum
04	a=DIP1-4, b=DIP5-8	LED1=borrow, 2-5=diff
05	a=DIP1-4, b=DIP5-8	LED1=콤, 2=같음, 3=작음

사용하지 않는 LED 비트는 0이다. reset하면 모드 01.

모드별 조작 6-10

모드	입력	출력
06	i=DIP1-4, s=DIP7-8	LED1=i[3-s]
07	i=DIP1, s=DIP6-8	s=0→LED1, s=7→LED8
08	DIP1-8 중 하나만 1	LED1-3: DIP1→0, DIP8→7
09	abc=DIP6,7,8	0→LED8, 7→LED1
10	hex=DIP5-8	a-dp 패턴, 1이면 점등

사용하지 않는 LED 비트는 0이다. reset하면 모드 01.

버튼을 길게 누르면

2단 동기화 → 안정된 입력 20클록 확인 → 상승 시 1클록 펄스.

1kHz에서 약 20ms 안정 시간에 동기화 지연이 추가된다.

누른 채 유지하면 한 번만 바뀐다. 떼 뒤 다시 눌러야 다음 모드가 된다.

모드 10 다음은 01. reset은 즉시 01로 돌아간다.

실험 영상에서 짧게 누르기·길게 누르기·순환을 각각 보여준다.

LCD가 표시되는 과정

전원 대기 50ms → 38,38,38,0C,06 초기 명령 → 두 줄 반복 갱신.

첫 줄 MODE 01...MODE 10, 두 번째 줄 회로 이름.

E high는 1ms, 바이트 간격은 최소 4ms. 첫 초기 명령 간격은 더 길다.

한 화면 시작 때 모드를 저장하여 두 줄이 다른 모드로 섞이지 않게 한다.

새 모드는 현재 전송을 마친 뒤 갱신되므로 잠깐 기다리고 읽는다.

CLI 실행 환경

macOS ARM64 바이너리 배포를 확인하고 WSL Ubuntu-26.04에서 Linux x86-64 바이너리를 실행했다.

제작자에게 Mac이 없으므로 Mac 실행·보드 기록을 검증했다고 쓰지 않는다.

API0 1.5.1, OSS CAD Suite 2026.08.19, openXC7 2026.08.20.

Yosys 0.63+173 / nextpnr-xilinx 68aeeb3 / Icarus Verilog 14.0 개발 버전.

openXC7 공식 ARM64·Linux 배포

설치와 경로

Python 3의 venv를 만들고 `pip install apio==1.5.1`을 실행한다.

`apio packages install`로 `oss-cad-suite`와 `openxc7`을 준비한다.

도구 경로는 `./apio/packages/oss-cad-suite/bin` 및 `openxc7/bin`이다.

`LD_LIBRARY_PATH`를 두 묶음의 `lib`로 전역 지정하지 않는다. 각 배포의 실행 래퍼가 자신의 라이브러리를 선택한다.

macOS는 `darwin-arm64`, WSL은 `linux-x86-64` 자산을 사용한다.

복사 가능한 전체 설치·실행 명령

대상 부품 데이터베이스

대상은 xc7s75fgga484-1이다. xc7s50으로 바꾸지 않는다.

배포에 포함된 S50 chipdb만으로는 S75를 빌드할 수 없다.

openXC7/prjxray-db의 spartan7 자료와 bbaexport.py·bbasm으로 정확한 부품의 chipdb를 준비한다.

데이터베이스 버전, 내보내기 결과와 실패 로그를 보관한다.
공식 디바이스 데이터베이스

실제 실행 결과와 남은 제약

Icarus: 2,560개 통과. Yosys: 논리 게이트·통합 합성 성공.

nextpnr: 논리 게이트 K4, 통합 클록 B6 핀을 찾지 못해 실패. 프레임·bit 단계는 실행하지 못했다.

검증한 S75 DB의 필수 핀 38개 중 28개가 빠져 있다. S75 tilegrid는 S50 파일과 바이트 단위로 같았다.

Vivado 공식 부품 데이터에는 K4와 B6가 존재한다. 다른 핀·다른 FPGA로 바꾸어 성공 처리하지 않는다.

CLI bit 목표는 미완료이며, 실행 로그·DB 해시·후속 작업을 공개한다. 실패 근거와 데이터베이스 대조

검증 가능한 순서로 실행

1. 02 Simulate: Icarus Verilog의 2,560개 검사와 VCD.
2. Yosys: synth_xilinx로 RTL을 Xilinx 셀에 매핑.
3. nextpnr-xilinx: 정확한 chipdb와 XDC로 배치·배선.
4. fasm2frames: FASM을 디바이스 프레임으로 변환.
5. xc7frames2bit: 정확한 part.yaml로 bit 생성.

각 단계의 입력·출력·종료 코드를 확인한다. 앞 단계 실패 후 오래된 산출물을 사용하지 않는다.

실행 명령과 결과 확인

프로젝트 폴더: opensource_cli/integrated.

```
python3 ../../tools/lab1.py simulate -project . -simulator iverilog
```

```
python3 ../../tools/openxc7_build.py -project .
```

생성 명령·실행 로그·상태·bit 해시는 프로젝트 build/cli와 검증 문서에서 확인한다.

합성 성공만으로 배치배선·bit 생성 완료를 선언하지 않는다.
실제 단계별 결과

대상 부품·핀 제약 (1)

xc7s75fpga484-1 · IOSTANDARD=LVC MOS33

포트	PACKAGE_PIN
clk	B6
rst	K4
mode_button	N8
sw[0]	U4
sw[1]	V4
sw[2]	W1

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (2)

xc7s75fgga484-1 · IOSTANDARD=LVCMOS33

포트	PACKAGE_PIN
sw[3]	W4
sw[4]	T1
sw[5]	U2
sw[6]	W3
sw[7]	Y1
led[0]	N5

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (3)

xc7s75fgga484-1 · IOSTANDARD=LVC MOS33

포트	PACKAGE_PIN
led[1]	M1
led[2]	M3
led[3]	M7
led[4]	N7
led[5]	M2
led[6]	M4

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (4)

xc7s75fgga484-1 · IOSTANDARD=LVC MOS33

포트	PACKAGE_PIN
led[7]	L4
seg_data[0]	R6
seg_data[1]	R4
seg_data[2]	R2
seg_data[3]	T5
seg_data[4]	N3

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (5)

xc7s75fgga484-1 · IOSTANDARD=LVCMOS33

포트	PACKAGE_PIN
seg_data[5]	P7
seg_data[6]	P3
seg_data[7]	P1
lcd_data[0]	A4
lcd_data[1]	B2
lcd_data[2]	C3

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (6)

xc7s75fgga484-1 · IOSTANDARD=LVCMOS33

포트	PACKAGE_PIN
lcd_data[3]	D4
lcd_data[4]	A2
lcd_data[5]	C5
lcd_data[6]	C1
lcd_data[7]	D1
lcd_e	A6

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (7)

xc7s75fgga484-1 · IOSTANDARD=LVCNMOS33

포트	PACKAGE_PIN
lcd_rs	G6
lcd_rw	D6

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

보드 실험으로 이어갈 때

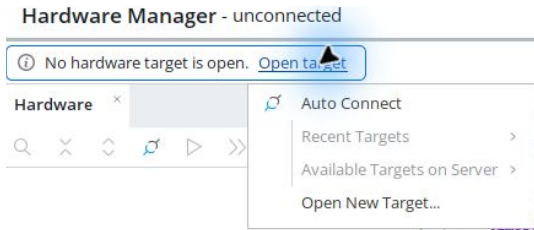
현재 검증한 CLI 흐름은 S75 데이터베이스의 핀 누락으로 bit를 생성하지 못했다.

보드 실험은 같은 RTL의 Vivado 통합 프로젝트에서 생성한 bit로 진행한다.

CLI 실행 결과와 Vivado bit의 생성 경로·해시를 구분하여 레포트에 적는다.

Vivado 통합 매뉴얼

실제 보드 연결



Open Hardware Manager → Open target → Auto Connect.

KEY1=reset, KEY2=다음 모드. DIP1-8=sw[7:0]. LED1-8=led[7:0].

장치가 없으면 보드 전원·USB JTAG·드라이버·VM USB 연결을 점검한다.

Program Device와 동작 확인

1. 연결된 장치 이름이 xc7s75인지 확인한다.
2. 장치 우클릭 → Program Device.
3. 이번 프로젝트의 lab1_integrated.bit를 선택하고 Program.
4. 기록 완료를 확인한 뒤 입력을 바꾸고 출력과 예상값을 비교한다.
5. 기록 성공 화면과 실제 보드 동작은 각각 증빙한다.

제작 환경의 실제 보드 기록·촬영 검증 여부는 검증표를 따른다. 파일 생성만으로 보드 동작 성공을 선언하지 않는다.

사진과 시연 영상 촬영

KEY1=reset, KEY2=다음 모드. DIP1-8=sw[7:0]. LED1-8=led[7:0].

사진에는 보드 연결과 입력·출력 위치가 함께 보이게 한다.

영상에는 입력을 조작하는 과정과 그에 따른 출력 변화를 담는다.

예상표의 정상·경계 조건을 직접 보여주고 회로 번호를 파일명에 적는다.

통합 영상이라면 각 회로의 타임스탬프를 레포트에서 연결한다.

GitHub 결과 정리

1. reports/pre와 reports/post에 실험 전·후 레포트를 둔다.
 2. evidence/vscode와 evidence/vivado에 로그·파형·캡처를 구분한다.
 3. evidence/board/photos와 videos에 직접 촬영한 자료를 둔다.
 4. 큰 영상·bit는 배포 위치를 정하고 README와 레포트에서 연결한다.
 5. 웹에서 사진 표시와 영상 재생 또는 다운로드가 되는지 확인한다.
- 레포트 양식·예시·GitHub 안내

실험 후 레포트

1. Vivado 버전·part·top·핀 제약·코드 커밋을 기록한다.
2. VS Code와 Vivado의 입력·출력·시간·검사 수를 비교한다.
3. 합성·구현·bit 경로와 경고·수정 사항을 설명한다.
4. 장치 기록 화면·사진·영상과 해당 조건을 연결한다.
5. 예상값·두 시뮬레이션·실측의 일치 또는 차이 원인을 해석한다.
6. 미수행 항목은 미완료로 남기고 후속 확인을 적는다.

전체 목차 PDF 프로젝트로 돌아가기